Ders 1 Genişletilmiş Notu: Algoritma Analizinin Temelleri ve Asimptotik Notasyonlar

Ders 1 Genişletilmiş Notu: Algoritma Analizinin Temelleri ve Asimptotik Notasyonlar

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Amacı ve Konuları
- 2. Algoritma Nedir, İyi Algoritma Nasıl Olur?
- 3. Temel Veri Yapıları ve Çalışma Zamanını Etkileyen Etkenler
- 4. Girdi Boyutu ve Çalışma Zamanı Birimi
- 5. Büyüme Mertebeleri
- 6. En Kötü, En İyi ve Ortalama Durum
- 7. Asimptotik Notasyonlar: Big O, Big Omega, Big Theta
- 8. Ardışık İki Parçadan Oluşan Algoritmalar
- 9. Büyüme Hızlarını Limitle Karşılaştırmak
- 10. Özyinelemesiz Algoritmaların Analizi
- 11. Özyinelemeli Algoritmaların Analizi
- 12. Alıştırmalar
- Örnek Sorular

1. Dersin Amacı ve Konuları

Hoca ilk derste üç şey yapacağını söyledi: dersin içeriğini tanıtmak, ilerleyen haftalarda neyle karşılaşılacağını anlatmak ve algoritma analizine temel bir giriş yapmak. Kaynak olarak slaytın başında verdiği kitaplardan özellikle ilk ikisini çok kullandığını, bunların çevrim içi olarak da bulunabildiğini belirtti.

Dersin amacı iki yönlüdür. Birincisi, temel algoritmik yapılar ve algoritma analizi üzerine lisansta edinilen bilgiyi tazelemektir. İkincisi, daha ileri (advanced) yapılara giriş yapmak ve bu yapıların bugün hangi amaçlarla kullanıldığını göstermektir. Hoca, özellikle ileri veri yapılarının çok çeşitli olduğunu, lisans seviyesinde bunları görme şansının pek olmadığını ve çeşitliliklerinin her yıl arttığını vurguladı. Dönem sonunda öğrencilerin bu yapılardan birini, kökenindeki temel yapıdan başlayarak bugünkü kullanım alanına kadar götüren bir sunum hazırlaması bekleniyor.

Slayttaki konu listesi şöyledir (ekrandan, 00:10:00):

- Algoritmaların temelleri (Fundamentals for Algorithms).
- Algoritma verimliliği analizinin temelleri ve asimptotik analiz: algoritma analizi, karmaşıklıklar, Big O, Big Theta, Big Omega, büyüme mertebeleri (orders of growth).
- Özyinelemesiz ve özyinelemeli algoritmaların analizi: çalışma zamanı, recurrence relation, backward substitution.
- Böl ve fethet (divide and conquer) algoritmalarının analizi: brute force, exhaustive search, decrease and conquer.
- Hashing algoritmaları.

- Dinamik programlama: rod cutting, matrix-chain multiplication, dinamik programlamanın öğeleri, optimal binary search trees.

Hoca listeyi sözlü olarak şöyle açtı. Asimptotik analizde verimlilik sınıflarına, Big O, Big Theta, Big Omega notasyonlarına ve büyüme hızı kavramına bakılacak. Özyinelemeli ve özyinelemesiz algoritmaların analizinde recurrence relation kurmak ve backward substitution gibi yöntemlerle çalışma zamanını kapalı bir formda yazmak öğrenilecek. Böl ve fethet algoritmalarında decrease and conquer ve transform and conquer gibi alt başlıklara da değinilecek; karşılaştırma yapabilmek için brute force ve exhaustive search algoritmaları da anlatılacak. Hashing'de klasik probing yöntemleri ve ardından kriptoloji açısından hashing ele alınacak. Dinamik programlamada hem çalışma mantığı hem analiz, rod cutting ve matrix chain multiplication örnekleriyle işlenecek. Sonra açgözlü (greedy) algoritmalar activity selection problemi üzerinden gelecek.

Dönemin ikinci yarısında amortize analiz anlatılacak. Hoca bunu, worst case ve best case analizinin biraz kötümser (pesimistik) olduğu fikrinden doğmuş bir analiz türü olarak tanıttı; potential method ve accounting method gibi yöntemleri vardır. Ardından B-tree ve Fibonacci heap gibi ileri veri yapıları, graf algoritmaları, temel string matching algoritmaları ve approximation algoritmaları gelecek.

2. Algoritma Nedir, İyi Algoritma Nasıl Olur?

Hoca algoritmayı şöyle tanımladı: belirsizliğe yol açmadan, yani "böyle mi, şöyle mi" gibi bir tereddüt bırakmadan, bir dizi komutun (instruction) net biçimde verilmesi ve problemin kabul edilebilir bir süre içinde, girdiyi alıp çıktıyı üretecek şekilde çözülmesi.

Bir algoritmanın iyi olması birkaç şeye bağlıdır:

- **Doğruluk:** Algoritma doğru çalışmalıdır.
- **Verimlilik (efficiency):** Bu dersin asıl ilgilendiği konudur. Verimlilik hem zaman hem yer (bellek, space) açısından değerlendirilir.
- **Genellik:** Sistemin gereksinimlerine göre mümkün olduğunca genelleştirilebilir, farklı alanlarda kullanılabilir, değiştirilebilir ve bakımı yapılabilir olmalıdır.

"Her zaman daha iyi bir algoritma var mıdır?" sorusunu araştırmak, hocanın deyişiyle bu bilimin işidir: daha iyi çalışma sürelerine sahip, optimuma daha yakın algoritmalar aranır.

3. Temel Veri Yapıları ve Çalışma Zamanını Etkileyen Etkenler

3.1 Temel veri yapıları

Hoca temel veri yapılarını hatırlattı ve ileri sürümlerine geçmeden önce her birinin temelini yeri geldikçe anlatacağını söyledi:

- Doğrusal yapılar: diziler, bağlı listeler, yığınlar (stack), kuyruklar (queue).
- Graflar: komşuluk (bitişiklik) matrisi ya da listelerle tutulurlar.
- Düğümlerden ve kenarlardan oluşan ağaçlar.
- Kümeler.

3.2 Performansı etkileyen etkenler

Bir algoritmanın performansını etkileyen parametreler iki gruba ayrılır. Sistemden bağımsız olan en büyük etken algoritmanın kendisidir, yani problemi çözerken seçtiğiniz yöntem. Sisteme bağlı etkenler ise donanım, derleyici gibi yazılımlar ve sistemin kendisidir. Bunlar birleşince algoritmanın gerçek çalışma süresi ortaya çıkar. Bu dersin konusu birinci gruptur.

4. Girdi Boyutu ve Çalışma Zamanı Birimi

Çalışma zamanını doğrudan etkileyen parametre, üzerinde işlem yapılan verinin boyutudur. Slayttaki örnekler (ekrandan, 00:15:20):

- İki matrisin çarpımında girdi boyutu, çarpılan matrislerdeki toplam eleman sayısı N'dir.
- Bir polinomu değerlendirirken girdi boyutu polinomun derecesidir.

Çalışma zamanı şu yaklaşık formülle ifade edilir:

$$T(n) \approx c_{op}\, C(n)$$

Burada n verinin boyutu, c_{op} algoritmadaki bir temel işlemin (basic operation; toplama, karşılaştırma gibi) bir kez çalışması için gereken süre, $C(n)$ ise bu temel işlemin algoritma boyunca kaç kez tekrarlandığıdır. c_{op} sisteme bağlı bir birimdir. İkisinin çarpımı algoritmanın toplam ne kadar zaman harcadığına dair bir bilgi verir. Analizi tamamlamak, temel işlemin tekrar sayısını n'e bağlı bir fonksiyon olarak bulmak demektir.

4.1 Örnek: Girdi iki katına çıkarsa

Slayttaki soru: $C(n) = \frac{1}{2}n(n-1)$ ise girdi boyutu iki katına çıktığında algoritma ne kadar uzun çalışır? (ekrandan, 00:15:20)

Önce $C(n)$ açılır:

$$C(n) = \frac{1}{2}n(n-1) = \frac{1}{2}n^2 - \frac{1}{2}n \approx \frac{1}{2}n^2$$

Hoca burada analizde hep akılda tutulan noktayı vurguladı: Önemli olan verinin küçük olduğu durumlar değil, çok büyük olduğu durumlarda olayın nereye gittiğidir. 10-15 elemanı sıralarken hangi algoritmayı kullandığınız ciddi bir fark yaratmaz; ama 1 milyon ya da 10 milyon elemandan söz ediyorsanız algoritma size hız kazandırır. Bu yüzden analiz hep $n \to \infty$ için yapılır. n'in çok büyük değerlerinde n^2 teriminin artışı çok büyük olduğundan, yalnız n katsayılı terimin etkisi ihmal edilecek kadar küçük kalır.

Oran:

$$\frac{T(2n)}{T(n)} \approx \frac{c_{op}C(2n)}{c_{op}C(n)} \approx \frac{\frac{1}{2}(2n)^2}{\frac{1}{2}n^2} = 4$$

Bu hesap şunu gösterir: c_{op} sadeleşir, yani sisteme bağlı bilgi algoritmadaki değişikliğin analizine bir şey katmaz; büyüme hızı (order of growth) açısından elenir. Bizim ilgilenmemiz gereken, veri iki kat, dört kat büyüdüğünde toplam çalışma süresinin kaç kat arttığıdır.

5. Büyüme Mertebeleri

Hoca sık karşılaşılan büyüme mertebelerini bir tablo üzerinden anlattı (ekrandan, 00:20:00):

Mertebe	Ad	Tipik kod yapısı	Açıklama	Örnek	$T(2N)/T(N)$
1	sabit	`a = b + c;`	tek ifade	iki sayıyı toplama	1
$\log N$	logaritmik	`while (N > 1) { N = N / 2; ... }`	yarıya bölme	ikili arama	~1
N	doğrusal	`for (int i = 0; i < N; i++) { ... }`	döngü	maksimumu bulma	2
$N \log N$	linearithmic	mergesort dersine bakınız	böl ve fethet	mergesort	~2
N^2	karesel	iki iç içe `for`	çift döngü	bütün çiftleri kontrol	4

Mertebe	Ad	Tipik kod yapısı	Açıklama	Örnek	$T(2N)/T(N)$
N^3	kübik	üç iç içe `for`	üçlü döngü	bütün üçlüleri kontrol	8
2^N	üstel	kombinatoryal arama dersine bakınız	exhaustive search	bütün alt kümeleri kontrol	

Hocanın anlatımıyla: Tek bir atama ya da iki sayıyı toplama gibi bir işlemin karmaşıklığı 1'dir; veriyi iki katına çıkarsanız bir etkisi olmaz. Bir dizide aranan elemanı her seferinde ikiye bölerek arıyorsanız (binary search) logaritmik bir karmaşıklık vardır ve veri iki katına çıktığında büyüme çok azdır. Diziyi baştan sona tarayıp minimum ya da maksimumu bulmak doğrusaldır; veri iki katına çıkınca süre de iki katına çıkar. Böl ve fethet algoritması olan mergesort $n \log n$'dir. İki ayrı dizideki eşleşen elemanları bulmak gibi iki iç içe döngülü bir algoritma karesel, matris çarpımı ya da üçlüleri belirleme kübiktir.

Hoca doğrusal ve $n \log n$ karmaşıklığın güzel, karesel karmaşıklığın belki kabul edilebilir olduğunu, ama bir noktadan sonra işlerin gerçekten karmaşıklaştığını ve verimli olabilmek için başka bir çözüm üretmek gerektiğini söyledi. Faktöriyel ve üstel karmaşıklıklı algoritmalar için nasıl bir optimizasyon yapılabileceğine bakmak gerekir.

6. En Kötü, En İyi ve Ortalama Durum

Bir algoritmanın analizinde, verinin dağılımına bağlı olarak iyi durum (best case), kötü durum (worst case) ve ortalama durum (average case) senaryoları vardır. Hoca lisansta genelde en kötü durumla çalışıldığını hatırlattı.

6.1 Sıralı arama örneği

Slayttaki algoritma (ekrandan, 00:25:40):

```
ALGORITHM SequentialSearch(A[0..n-1], K)
//Searches for a given value in a given array by sequential search
//Input: An array A[0..n-1] and a search key K
//Output: The index of the first element in A that matches K
//        or -1 if there are no matching elements
i <- 0
while i < n and A[i] != K do
    i <- i + 1
if i < n return i
else return -1
```

Dizinin sonuna gelinmediği ve eleman bulunmadığı sürece arama devam eder. Döngüden çıkıldığında dizinin sonuna gelinmişse eleman bulunamamıştır ve -1 döner; aksi hâlde i döner. Temel işlem, döngüdeki kontrol ile birlikte yapılan artırmadır.

- **En kötü durum:** Aranan eleman bulunamaz ya da en sonda bulunur. Bütün elemanlar tek tek taranır: $C_{worst}(n) = n$.
- **En iyi durum:** Eleman ilk konumda bulunur, tek kontrol yapılır: $C_{best}(n) = 1$.

Hoca bazı algoritmalarda en kötü, bazılarında en iyi, bazılarında ise ortalama durumu kullanmanın mantıklı olabileceğini, bunun verinin ve problemin durumuna bağlı olduğunu söyledi. Ortalama durumu hesaplamak ise bir tık daha zordur.

6.2 Ortalama durumun hesabı

Ortalama durum için iki varsayım yapılır (ekrandan, 00:25:40):

- (a) Başarılı arama olasılığı p'dir ($0 \leq p \leq 1$). Yani dizi baştan sona tarandığında aranan elemanı bulma olasılığı.
- (b) İlk eşleşmenin i. konumda olma olasılığı her i için aynıdır. Hoca bunu verinin düzgün (uniform) dağıldığı varsayımı olarak anlattı.

Başarılı aramada eleman birinci konumda bulunursa olasılık p/n'dir ve 1 adım yapılır, katkısı $1 \cdot \frac{p}{n}$'dir. İkinci konumda bulmak için döngü iki kez döner, katkı $2 \cdot \frac{p}{n}$'dir. Bu son elemana kadar sürer: $n \cdot \frac{p}{n}$. Köşeli parantezin içi, elemanın bulunduğu durumların ortalama tekrar sayısıdır. Bulunamama olasılığı $1 - p$'dir ve bulunamazsa döngü n kez döner; bu da $n(1-p)$ olarak eklenir:

$$C_{avg}(n) = \left[1 \cdot \frac{p}{n} + 2 \cdot \frac{p}{n} + \cdots + i \cdot \frac{p}{n} + \cdots + n \cdot \frac{p}{n}\right] + n(1-p)$$

Köşeli parantezdeki toplam $\frac{p}{n}(1 + 2 + \cdots + n) = \frac{p}{n} \cdot \frac{n(n+1)}{2}$ olduğundan (ara adım eklendi):

$$C_{avg}(n) = \frac{p(n+1)}{2} + n(1-p)$$

Bu kapalı form derste yazılmadı; python ile ($n = 7$, $p = 1/3$ için iki ifade de 6) doğrulandı.

Hocanın yorumu: Dizi düzgün dağılımlıysa ve arama başarılıysa, aranan eleman ortalamada dizinin ortalarında bulunur, yani yaklaşık $n/2$ karmaşıklığa gidilir. p'nin 0 ile 1 arasındaki değerine göre sonuç en iyi ya da en kötü duruma doğru evrilir. Bu yüzden p'nin verinin dağılımına göre gerçekçi seçilmesi önemlidir. Örneğin Gauss dağılımlı bir veride aramanın ortalarda başarılı olma ihtimali daha yüksek olabilir. Dağılımı bölüm bölüm değişen bir veride ise her bölüm için ayrı bir olasılık varsayıp ayrı ayrı hesap yapmak mantıklı olabilir.

> **Ek bilgi (derste anlatılmadı):** Formülün iki uç durumu sezgiyi doğrular. $p = 1$ (arama kesin başarılı) için $C_{avg}(n) = \frac{n+1}{2}$, yani yaklaşık dizinin yarısı taranır. $p = 0$ (arama kesin başarısız) için $C_{avg}(n) = n$, yani bütün dizi taranır.

7. Asimptotik Notasyonlar: Big O, Big Omega, Big Theta

Bir algoritmanın karmaşıklığı, çalıştığı veriye bağlı olarak büyüme sınıflarından birine ya da birkaçının birleşimine ait olabilir. Ait olduğu sınıfı bulmak için Big O, Big Omega ve Big Theta notasyonları kullanılır. Big O algoritmanın üst sınırını (upper bound) belirler: karmaşıklık "şunun altındadır" denir. Big Omega alt sınırı (lower bound) belirler: "şunun üstündedir" denir. İkisinin de gerçek karmaşıklığa mümkün olduğunca yakın olduğu durumu Big Theta verir; hoca ve slayt buna "average bound" adını verdi.

Hoca slayttaki sıralamayı da gösterdi (ekrandan, 00:40:00):

$$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < \cdots < 2^n < 3^n < \cdots < n^n$$

7.1 Big O

$f(n)$ analiz edilen algoritmanın çalışma zamanı fonksiyonudur. $f(n) \in O(g(n))$ demek için, pozitif bir c sabiti ve negatif olmayan bir n_0 tam sayısı bulunmalı ve her $n \geq n_0$ için

$$f(n) \leq c\, g(n)$$

sağlanmalıdır. "$O(n)$" ya da "$O(n^2)$" derken oradaki n ya da n^2, $g(n)$'e karşılık gelir.

Slayttaki örnek (ekrandan, 00:35:30): Slayt fonksiyonu `2n*3` diye yazmış, hoca da "2n çarpı 3" diye okudu. Sağ tarafa uygun bir $c\, g(n)$ yazmak yeterlidir; böyle bir küme bulmak yeter.

- $2n + 3 \leq 7n$, $n \geq 1$: sağlanır ("ok"). Burada $c = 7$, $g(n) = n$.
- $2n + 3 \leq 2n + 3n = 5n$, $n \geq 1$: sağlanır. $c = 5$, $g(n) = n$. Sonuç: $f(n) \in O(n)$.

- $2n + 3 \leq 4n^2 + 3n^2 = 7n^2$, $n \geq 1$: sağlanır. $c = 7$, $g(n) = n^2$. Sonuç: $f(n) \in O(n^2)$.

  Ders sonrası not: Slaytta ve hocanın okumasında fonksiyon "$2n \cdot 3$" olarak geçiyor; doğrusu $2n + 3$'tür. $2n \cdot 3 = 6n$ olsaydı $6n \leq 5n$ eşitsizliği hiçbir $n \geq 1$ için sağlanmazdı. Aynı fonksiyon Theta slaytında $f(n) = 2n + 3$ olarak yazılıdır ve "$2n + 3n$" satırı da bunu gösterir.

Hocanın vurguladığı nokta: $O(n)$ de $O(n^2)$ de doğrudur, çünkü Big O bir sınır belirler: "n'in ya da n^2'nin altında bir yerde" demiş olursunuz. $8n^3$ deseydiniz onun altında olduğunu söylemiş olurdunuz. Hepsi fonksiyonun yeri hakkında doğru bilgi verir, ama sınırı yukarı çektikçe gerçek karmaşıklıktan uzaklaşırsınız. Bu fonksiyon için $O(n)$, $O(n^2)$, $O(2^n)$ hepsi yazılabilir; üst sınır kümesi n dahil yukarı doğru giden sınıflardır. $O(\log n)$ ise yazılamaz, çünkü hiçbir c için $c \log n$ değeri $f(n)$'den büyük kalmaz. Yani n'den aşağıdaki sınıflar üst sınır olamaz.

7.2 Big Omega

Tanım aynıdır, yalnız eşitsizliğin yönü değişir (ekrandan, 00:38:20): $f(n) \in \Omega(g(n))$ için pozitif bir c ve negatif olmayan bir n_0 bulunmalı ve her $n \geq n_0$ için

$$f(n) \geq c\, g(n)$$

sağlanmalıdır.

Aynı fonksiyon için:

- $2n + 3 \geq 1 \cdot n$, her $n \geq 1$: sağlanır, $f(n) \in \Omega(n)$.
- $2n + 3 \geq \log n$, her $n \geq 1$ ($c = 1$): sağlanır, $f(n) \in \Omega(\log n)$.
- $f(n) \in \Omega(n^2)$: sağlanmaz ("not ok"); eşitsizliği sağlayan bir (c, n_0) ikilisi bulunamaz.

7.3 Big Theta

Alt ve üst sınırın arasında kalan kısım, yani ikisinin kesiştiği, gerçeğe en yakın tespit Big Theta'dır (ekrandan, 00:40:00). $f(n) \in \Theta(g(n))$ için pozitif c_1, c_2 sabitleri ve negatif olmayan bir n_0 bulunmalı ve her $n \geq n_0$ için

$$c_1\, g(n) \leq f(n) \leq c_2\, g(n)$$

sağlanmalıdır. Önemli olan, $g(n)$'in iki eşitsizlik parçasında da aynı olmasıdır. Özetle bir alt sınır ve bir üst sınır tespit etmiş olursunuz.

Örnek: $1 \cdot n \leq 2n + 3 \leq 5 \cdot n$. Burada $c_1 = 1$, $c_2 = 5$, $g(n) = n$; sonuç $f(n) = \Theta(n)$.

7.4 Örnek: $\frac{1}{2}n(n-1) \in \Theta(n^2)$ mi?

Hem üst hem alt sınır bulunmalıdır.

Üst sınır: $\frac{1}{2}n^2 - \frac{1}{2}n \leq c_2\, g(n)$ olacak bir c_2 aranır. $c_2 = \frac{1}{2}$ ve $g(n) = n^2$ seçilirse, sol tarafta bir çıkarma işlemi olduğu için eşitsizlik sağlanır.

Alt sınır: Aynı $g(n) = n^2$ ile $c_1\, n^2 \leq \frac{1}{2}n^2 - \frac{1}{2}n$ olacak bir c_1 aranır. $c_1 = 1$ sağlanmaz; $c_1 = 2$ de sağlanmaz, çünkü sağ tarafta $\frac{1}{2}n^2$'den bir şey çıkarılıyor. c_1 küçüldükçe, $\frac{1}{4}$, $\frac{1}{8}$ gibi değerlerde eşitsizlik sağlanmaya başlar. $c_1 = \frac{1}{4}$ seçilir.

n_0'ın varlığı (ekrandan, 00:43:00): Bulunan katsayılar arasına fonksiyon yazılır:

$$\frac{1}{4}n^2 \leq \frac{1}{2}n^2 - \frac{1}{2}n \leq \frac{1}{2}n^2$$

- $n_0 = 1$: sol taraf $\frac{1}{4}$, orta $\frac{1}{2} - \frac{1}{2} = 0$, sağ $\frac{1}{2}$. Orta değer $\frac{1}{4}$ ile $\frac{1}{2}$ arasında kalmaz: olmaz ("Not ok").
- $n_0 = 2$: sol 1, orta $2 - 1 = 1$, sağ 2. $1 \leq 1 \leq 2$: olur ("Ok", slaytta "better" notuyla).

- $n_0 = 3$: sol 2.25, orta $4.5 - 1.5 = 3$, sağ 4.5. $2.25 \leq 3 \leq 4.5$: olur.

Demek ki $n \geq 2$ için tutarlı bir eşitsizlik vardır ve $\frac{1}{2}n(n-1) \in \Theta(n^2)$'dir. (Değerler python ile kontrol edildi.)

7.5 Baskın terim

Hoca $4n^2+5n$ biçiminde bir fonksiyonla devam etti. Burada bir baskınlık (dominasyon) vardır: Bir fonksiyonda n'in farklı kuvvetleri birleşmişse, n'in büyük değerlerinde n^2'nin yükselme hızı çok fazla olduğundan bu terim küçük kuvvetleri domine eder. Bu yüzden ilk bakışta da bir çıkarım yapılabilir. Big Omega için $c_1 = 1$, $g(n) = n^2$; Big O için $c = 9$, $g(n) = n^2$ verildiğinde eşitsizlikler sağlanır. Sonuç: $\Theta(n^2)$.

7.6 Örnek: $f(n) = n^2 \log n + n$

Slayttaki çözüm (ekrandan, 00:45:30):

$$n^2 \log n \leq n^2 \log n + n \leq 5\, n^2 \log n$$

Burada $g(n) = n^2 \log n$, katsayılar 1 ve 5'tir. Böylece $\Omega(n^2 \log n)$, $O(n^2 \log n)$ ve $\Theta(n^2 \log n)$ birbiriyle çakışan sınırlar olarak bulunur. Hoca bu fonksiyonun karmaşıklık sınıfının sıralamada n^2 ile n^3 arasında bir yere karşılık geldiğini söyledi; slayttaki ok da bu aralığı gösteriyor.

> **Ders sonrası not:** $\log 1 = 0$ olduğundan bu eşitsizlik $n = 1$'de sağlanmaz ($1 \leq 0$ olmaz); $\log_2$ ile $n_0 = 2$ alınabilir. Python ile kontrol edildi.

7.7 Grafik yorum

Hoca "her yerde görülen meşhur grafiği" anlattı. Yatay eksen verinin boyutu, dikey eksen büyüme hızıdır. $t(n)$ fonksiyonu, $c_1 g(n)$ ile $c_2 g(n)$ eğrileri arasında kalır. n_0'ı belirlemek bu yüzden önemlidir: n'in küçük değerleri için değil, belirli bir noktadan sonraki değerleri için bu sınıfsal yapı oturur. Ama n_0'ın tam olarak nerede olduğuyla doğrudan ilgilenilmez; n_0'dan büyük değerlerde tutarlılığın sağlandığını görmek yeterlidir. Alt ve üst sınırlar birbirine ne kadar yaklaştırılırsa analiz o kadar gerçekçi olur.

8. Ardışık İki Parçadan Oluşan Algoritmalar

Arka arkaya çalışan iki parçanın karmaşıklıkları biliniyorsa, bütün algoritmanın karmaşıklığı hakkında ne söylenebilir? Slayttaki teorem (ekrandan, 01:04:40):

Teorem: $t_1(n) \in O(g_1(n))$ ve $t_2(n) \in O(g_2(n))$ ise

$$t_1(n) + t_2(n) \in O(\max\{g_1(n), g_2(n)\})$$

Bu, daha önce söylenen "n^3, n^2'yi domine eder" fikrinin teoremidir.

8.1 Kanıt

Önce bir yardımcı önerme: a_1, b_1, a_2, b_2 dört sayı olsun. $a_1 \leq b_1$ ve $a_2 \leq b_2$ ise $a_1 + a_2 \leq 2\max\{b_1, b_2\}$. Yani iki küçük sayının toplamı, büyüklerin en büyüğünün iki katını geçmez.

Big O tanımından, iki fonksiyon için ayrı sabitler ve ayrı eşik değerler vardır:

$$t_1(n) \leq c_1 g_1(n) \quad (n \geq n_1), \qquad t_2(n) \leq c_2 g_2(n) \quad (n \geq n_2)$$

$c_3 = \max\{c_1, c_2\}$ alınsın ve $n \geq \max\{n_1, n_2\}$ olsun. Bu durumda iki eşitsizlik birlikte geçerlidir ve toplanır:

$$t_1(n) + t_2(n) \leq c_1 g_1(n) + c_2 g_2(n)$$

c_1 ve c_2 yerine daha büyük olan c_3 konursa eşitsizliğin yönü bozulmaz:

$$\leq c_3 g_1(n) + c_3 g_2(n) = c_3[g_1(n) + g_2(n)]$$

Yardımcı önermeyle köşeli parantez, büyüğün iki katıyla sınırlanır:

$$\leq c_3 \cdot 2 \max\{g_1(n), g_2(n)\}$$

Böylece $t_1(n) + t_2(n) \in O(\max\{g_1(n), g_2(n)\})$ bulunur.

8.2 Örnek: Dizide aynı elemanları bulmak

Bir dizide birbirine eşit elemanlar olup olmadığına bakılıyor. Hoca bilerek "kötü" bir algoritma kurdu: önce diziyi sırala, sonra sıralanmış dizide yan yana elemanlara bak. Sıralama $\frac{1}{2}n(n-1)$ karşılaştırmayla yapılırsa n^2 mertebesindedir; ardından yan yana elemanları taramak $n-1$ karşılaştırmadır. Teoreme göre algoritmanın karmaşıklığı ikisinin büyüğü, yani n^2'dir.

Hoca sonra öğrencilere bu problemin normalde Big O'da kaçta çözüleceğini sordu ve şu cevabı verdi: Her elemanın tekrar sayısını tutan ek bir yapı olursa problem $O(n)$'de çözülebilir. Bu durumda bellek karmaşıklığı artar: şu an yalnız n elemanlık bir blok varken ek yapıyla yaklaşık $2n$'lik bir yer gerekir, ama süre n'e iner. Zaman ile bellek arasında bir takas vardır.

> **Ders sonrası not:** Ek bellekle doğrusal süre büyük olasılıkla ancak değer aralığı küçükse (sayma dizisi) ya da hash tablosuyla (beklenen süre olarak) elde edilir. Yalnız karşılaştırmaya dayalı genel çözümde bilinen iyi yol, $O(n \log n)$'lik bir sıralama algoritmasıyla sıralayıp yan yana bakmaktır.

9. Büyüme Hızlarını Limitle Karşılaştırmak

Farklı algoritmaların büyüme hızlarını karşılaştırmanın bir yolu limit kullanmaktır. Bir algoritmanın büyüme hızıyla ilgili bir tahmininiz varsa onu doğrulamak için de kullanılabilir. $t(n)$ ve $g(n)$ için $n \to \infty$'da oranın limitine bakılır:

- Limit sabit bir sayıysa: $t(n) \in \Theta(g(n))$, yani aynı büyüme mertebesi.
- Limit 0 ise: $t(n) \in O(g(n))$; t daha yavaş büyür.
- Limit sonsuzsa: $t(n) \in \Omega(g(n))$; t daha hızlı büyür.

9.1 Örnek: $\frac{1}{2}n(n-1)$ ve n^2

Slayttaki hesap (ekrandan, 01:10:00):

$$\lim_{n\to\infty} \frac{\frac{1}{2}n(n-1)}{n^2} = \frac{1}{2} \lim_{n\to\infty} \frac{n^2 - n}{n^2} = \frac{1}{2} \lim_{n\to\infty} \left(1 - \frac{1}{n}\right) = \frac{1}{2}$$

Sabitler limitin dışına çıkarılır, pay açılır, sadeleştirilir. Limit sabit bir sayıya gittiği için iki fonksiyon aynı büyüme mertebesindedir: $\frac{1}{2}n(n-1) \in \Theta(n^2)$. Slayt ters oranı da soruyor: $\lim \frac{n^2}{\frac{1}{2}n(n-1)}$. Hoca bunun da sabit bir sayı vereceğini ve aynı sonucu doğrulayacağını söyledi (limit 2'dir).

9.2 Örnek: $n!$ ve 2^n

$n!$'in açılımı için Stirling formülü kullanılır; n'in çok büyük değerlerinde yakınsayan bir eşitliktir ve burada sonsuza giden durum konuşulduğu için kullanışlıdır (ekrandan, 01:11:40):

$$n! \approx \sqrt{2\pi n}\left(\frac{n}{e}\right)^n$$

Yerine konur:

$$\lim_{n\to\infty}\frac{n!}{2^n} = \lim_{n\to\infty}\frac{\sqrt{2\pi n}\,(n/e)^n}{2^n} = \lim_{n\to\infty}\sqrt{2\pi n}\,\frac{n^n}{2^n e^n} = \lim_{n\to\infty}\sqrt{2\pi n}\left(\frac{n}{2e}\right)^n = \infty$$

Limit sonsuza gittiği için $n! \in \Omega(2^n)$'dir. Hocanın yorumu: 2^n oldukça hızlı büyür, ama $n!$ çok daha hızlı büyür.

> **Ders sonrası not:** Hoca bir noktada "$n!$ Big O'da 2^n yazabiliriz" dedi (01:12:18). Slayttaki ve doğru sonuç $n! \in \Omega(2^n)$'dir; $n!$ daha hızlı büyüdüğü için $O(2^n)$'de değildir.

Slayttaki soru, limit yönteminin neden daha kesin olduğunu gösterir: Big Omega notasyonu $n!$ ile 2^n'in aynı büyüme mertebesinde olma ihtimalini dışlayabilir mi? Hocanın cevabı: Asimptotik notasyondaki "büyük eşit" işareti yüzünden alt ve üst sınır belirlendiğinde eşitlik de sınırın içine konmuş olur; bu yüzden yalnız Ω ile eşit olma durumu elenemez. Limit yönteminde ise aynı mertebede olsalardı limit sabit bir değer verirdi; limit sonsuz çıktığı için eşitlik elenir. Yani limit yöntemi bir tık daha kesin bilgi verir.

10. Özyinelemesiz Algoritmaların Analizi

10.1 MaxElement

Hoca ilk olarak dizideki en büyük elemanı bulan algoritmanın sözde kodunu gösterdi. n elemanlı bir dizi alınır, bir `maxval` tanımlanır ve dizi boyunca gezilerek yerel maksimumdan genel maksimuma geçilir. Özyinelemesiz (sıralı) algoritmalarda temel yol şudur: bir temel işlem belirlenir ve tekrar sayısı formülleştirilir; toplam sembolleri bu işte çok etkilidir. Bu örnekte temel işlem döngüdeki karşılaştırmadır ve i 1'den $n-1$'e giden döngüde bir kez yapılır:

$$C(n) = \sum_{i=1}^{n-1} 1 = n - 1 \in \Theta(n)$$

10.2 Genel plan

Hoca özyinelemesiz algoritmaların analizi için adımları sıraladı:

- Girdi boyutunu gösteren parametreye karar ver. Karmaşıklık hangi parametreye göre hesaplanacak?
- Temel işlemi belirle. Hangisinin tekrar sayısı sayılacak?
- Temel işlemin çalışma sayısı yalnız girdi boyutuna mı bağlı, kontrol et. Bu, en kötü, en iyi ya da ortalama durumu kullanmaya yönelmenizi sağlar.
- Temel işlemin kaç kez çalıştığını gösteren bir toplam ifadesi kur.
- Standart formüllerle büyüme mertebesini bul.

Hoca standart toplam formüllerini kitapların ek (appendix) kısımlarında bulunabileceğini, kolaylık olsun diye slaytlara da eklediğini söyledi. Sabitlerin toplamın dışına çıkarılması, toplamdaki farklı terimlerin ayrılması ve alta/üste yuvarlama kuralları sık kullanılacak.

10.3 UniqueElements

Dizideki bütün elemanların birbirinden farklı olup olmadığını kontrol eden algoritma (ekrandan, 01:20:00):

```
ALGORITHM UniqueElements(A[0..n-1])
//Determines whether all the elements in a given array are distinct
//Input: An array A[0..n-1]
//Output: Returns "true" if all the elements in A are distinct
//        and "false" otherwise
for i <- 0 to n-2 do
    for j <- i+1 to n-1 do
        if A[i] = A[j]
            return false
return true
```

İki iç içe döngü karşılıklı elemanlara bakar. Eşit bir çift bulunursa `false` döner, çünkü bütün elemanların farklı olması koşulu sağlanmamıştır; hiç eşit çift bulunamazsa her şey yolundadır ve `true` döner. Temel işlem eşitlik kontrolüdür. Analiz en kötü durum için yapılır: Algoritma ilk iki elemanın eşit olduğunu bulup hemen çıkabilirdi, ama en kötü durumda bütün çiftleri taraması gerekir. İç döngü $j = i + 1..n - 1$, dış döngü $i = 0..n - 2$'dir:

$$C_{worst}(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} (n - 1 - i) = \frac{(n-1)n}{2} \approx \frac{1}{2}n^2 \in \Theta(n^2)$$

İç toplam açılırken üst sınırdan alt sınır çıkarılıp bir eklenir. Hoca $\frac{n^2}{2}$'deki "bölü 2"nin sezgisini şöyle verdi: Taranan alanı görselleştirirseniz, bir karenin köşegeninin üst tarafındaki üçgeni taramış olursunuz. İç içe iki döngü gördüğünüzde zaten aşağı yukarı n^2 beklenir.

10.4 Matris çarpımı

İki kare matrisin çarpımında, birincinin satırlarıyla ikincinin sütunları ilk iki döngüde eşleşir; üçüncü döngü karşılıklı elemanları çarpıp toplar. Temel işlem bu çarpma ve eklemedir. Üç döngü de 0'dan $n - 1$'e gittiği için sonuç n^3'tür:

$$M(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1 = n^3$$

10.5 İkilik gösterimdeki basamak sayısı

Onluk tabanda verilen pozitif bir sayının ikilik (binary) gösterimindeki basamak sayısını bulmak için sayı her seferinde ikiye bölünür ve kaç kez bölünebildiğine bakılır. Her adımda yarıya inildiği için döngüye giriş sayısı logaritmiktir. İkinin kuvveti olmayan sayılarda tam dönüş sayısı için alta yuvarlama gerekir. Ayrıca `while` döngüsünden son kez çıkmadan önce bir karşılaştırma daha yapılır ($n = 1$ olduğunda); $+1$ buradan gelir. Sonuç:

$$C(n) = \lfloor \log_2 n \rfloor + 1 \in \Theta(\log n)$$

11. Özyinelemeli Algoritmaların Analizi

11.1 Faktöriyel ve recurrence relation

Negatif olmayan n için $n!$'i hesaplayan özyinelemeli algoritmada hoca önce özyinelemenin mantığını hatırlattı: Özyinelemeli bir algoritma yazarken mutlaka bir çıkış durumu (base case) olmalıdır. Burada $n = 0$ olduğunda 1 döndürülür. Diğer durumlarda fonksiyon $n-1$ için kendini çağırır ve sonucu n ile çarpar: $F(n) = F(n - 1) \cdot n$.

Analizde recurrence relation kurulur. Hoca bunun fonksiyonun sonucunu üreten mekanizmayı değil, işlem sayısını n'e bağlı olarak verdiğini vurguladı. $n!$'i hesaplamak için gereken çarpma sayısı, $(n-1)!$'i hesaplamak için gereken çarpma sayısına, sonunda $(n-1)!$ ile n'i çarpan bir çarpma eklenerek bulunur:

$$M(n) = M(n-1) + 1, \quad n > 0, \qquad M(0) = 0$$

Bu ifadenin sağ tarafında hâlâ $M(n-1)$ vardır; sonucu yalnız n cinsinden bulmak için recurrence çözülmelidir. Yöntemlerden en basiti **backward substitution**'dır: belirli sayıda adım geriye giderek recurrence'ın aldığı değerler gözlenir, gidişat hakkında bir görüş oluşturulur ve genel bir formüle dönüştürülür.

$$M(n) = M(n-1) + 1$$

Bir adım geri: $M(n-1) = M(n-2) + 1$. Yerine konur:

$$M(n) = [M(n-2) + 1] + 1 = M(n-2) + 2$$

Bir adım daha: $M(n-2) = M(n-3) + 1$:

$$M(n) = [M(n-3) + 1] + 2 = M(n-3) + 3$$

i adım geri gidildiğinde:

$$M(n) = M(n-i) + i$$

Şimdi başlangıç koşulu (initial condition) kullanılır: Çağrılar $n = 0$'da durur ve $M(0) = 0$'dır. $n - i = 0$ olması için $i = n$ alınır:

$$M(n) = M(n-n) + n = M(0) + n = n \in \Theta(n)$$

11.2 Özyinelemeli algoritmalar için genel plan

Slayttaki plan (ekrandan, 01:30:00):

- Girdi boyutunu gösteren parametreye (ya da parametrelere) karar ver.
- Algoritmanın temel işlemini belirle.
- Temel işlemin çalışma sayısının aynı boyuttaki farklı girdilerde değişip değişmediğini kontrol et. Hoca bunun hesabın hangi duruma (en kötü, en iyi, ortalama) göre yapılacağına karar vermeyi sağladığını söyledi.
- Temel işlemin çalışma sayısı için uygun bir başlangıç koşuluyla bir recurrence relation kur.
- Recurrence'ı çöz ya da en azından çözümünün büyüme mertebesini bul.

11.3 Hanoi kuleleri

n diskin, her zaman küçük disk büyük diskin üstünde kalacak şekilde, bu kuralı bozmadan, kaynak çubuktan hedef çubuğa taşınmasıdır. Hoca bunun aslında bir oyuncak olduğunu söyledi. Çözüm özyinelemelidir. n disk iki parçaya ayrılır: tabandaki en büyük disk ve onun üstündeki $n-1$ disk. Çubuklara A, B, C denirse ve kaynak A, hedef C ise:

- Üstteki $n-1$ disk, C yardımcı olarak kullanılarak A'dan B'ye taşınır. Bu kendi içinde özyinelemeli bir iştir.
- Açıkta kalan en büyük disk tek hamlede A'dan C'ye konur.
- B'deki $n-1$ disk, bu kez boşta kalan A kullanılarak C'ye taşınır.

Yani iki tane $n-1$ disklik özyinelemeli transfer ve bir tane tek hamle vardır. Tek disk kaldığında tek hamlede yerleştirilir; sonlanma noktası budur. Recurrence (ekrandan, 01:34:30):

$$M(n) = 2M(n-1) + 1, \quad n > 1, \qquad M(1) = 1$$

Backward substitution:

$$M(n) = 2M(n-1) + 1 \qquad \text{(yerine koy: } M(n-1) = 2M(n-2) + 1\text{)}$$

$$= 2[2M(n-2) + 1] + 1 = 2^2 M(n-2) + 2 + 1 \qquad \text{(yerine koy: } M(n-2) = 2M(n-3) + 1\text{)}$$

$$= 2^2[2M(n-3) + 1] + 2 + 1 = 2^3 M(n-3) + 2^2 + 2 + 1$$

Katsayılardaki düzen fark edilince i'ye bağlı genel ifade yazılır:

$$M(n) = 2^i M(n-i) + 2^{i-1} + 2^{i-2} + \cdots + 2 + 1 = 2^i M(n-i) + 2^i - 1$$

Başlangıç koşulu $n = 1$'dir; $M(n-i)$'nin içinin 1 olması için $i = n - 1$ alınır:

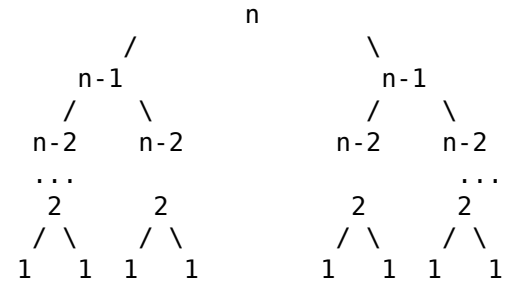
$$M(n) = 2^{n-1} M(n-(n-1)) + 2^{n-1} - 1 = 2^{n-1} M(1) + 2^{n-1} - 1 = 2^{n-1} + 2^{n-1} - 1 = 2^n - 1$$

Sonuç $\Theta(2^n)$'dir. ($M(n) = 2^n - 1$, python ile $n = 1..9$ için doğrulandı.)

11.4 Özyinelemeli çağrı ağacı

Backward substitution dışında başka yöntemler de vardır; biri ağaç üzerinden çözümdür. Hoca bunun görsel ve, ağaca dökülebiliyorsa, kolay bir yol olduğunu söyledi. Ağaçta düğümler özyinelemeli çağrılara karşılık gelir (ekrandan, 01:36:50).

Hanoi'de n için $n-1$'e iki çağrı yapılır ve her çağrıda bir işlem ($+1$) vardır. Ağaç şöyle büyür:



Her düğümde bir işlem yapıldığından toplam işlem sayısı düğüm sayısına eşittir. Ağacın seviyesi l, 0'dan $n-1$'e gider ve her seviyede 2^l düğüm vardır:

$$C(n) = \sum_{l=0}^{n-1} 2^l = 2^n - 1$$

Böylece backward substitution ile bulunan sonuç ağaç üzerinden de bulunmuş olur. Hoca dikkat edilecek noktayı şöyle özetledi: Recurrence relation'da her n boyutu için kaç kez fonksiyon çağrısı yapıldığı ve her çağrıda ayrıca kaç işlem yapıldığı ağaç üzerinde işaretlenir. Her çağrıdaki işlem sayısının 1'den farklı olduğu durumların ilerleyen haftalarda çözüleceğini söyledi.

11.5 Özyinelemeli ikilik basamak sayısı

10.5'teki problemin özyinelemeli sürümünde fonksiyon $n > 1$ iken kendini $\lfloor n/2 \rfloor$ ile çağırır ve dönüş değerini bir artırır. $n = 1$'de toplama yapılmaz:

$$A(n) = A(\lfloor n/2 \rfloor) + 1, \quad n > 1, \qquad A(1) = 0$$

Bölmeyi ve aşağı yuvarlamayı çözüme basitçe katabilmek için $n = 2^k$ alınır (ekrandan, 01:40:00). $n/2$, 2^{k-1} olarak yazılır:

$$A(2^k) = A(2^{k-1}) + 1, \quad k > 0, \qquad A(2^0) = 0$$

Backward substitution:

$$A(2^k) = A(2^{k-1}) + 1 \qquad \text{(yerine koy: } A(2^{k-1}) = A(2^{k-2}) + 1)$$

$$= [A(2^{k-2}) + 1] + 1 = A(2^{k-2}) + 2 \qquad \text{(yerine koy: } A(2^{k-2}) = A(2^{k-3}) + 1)$$

$$= [A(2^{k-3}) + 1] + 2 = A(2^{k-3}) + 3$$

$$\cdots = A(2^{k-i}) + i$$

Başlangıç durumunu sağlamak için ikinin üssü sıfır olmalıdır, yani $i = k$:

$$A(2^k) = A(1) + k = k$$

$n = 2^k$ olduğundan $k = \log_2 n$'dir:

$$A(n) = \log_2 n \in \Theta(\log n)$$

Hoca backward substitution'ı şöyle özetledi: Her seferinde bir adım geri gidersiniz, yerine koyarsınız, bir düzen fark edersiniz, i'ye bağlı yazarsınız; sonra başlangıç durumu için gereken değeri i terimini yok edecek şekilde yerleştirip işlemi tamamlarsınız.

11.6 Fibonacci: backward substitution'ın zorlandığı yer

Backward substitution her zaman iyi çalışmaz. Fibonacci sayılarında ardışık iki sayının toplamı bir sonrakini verir: $0 + 1 = 1$, $1 + 1 = 2$, $2 + 1 = 3$ gibi. Özyinelemeli tanım:

$$F(n) = F(n-1) + F(n-2)$$

İki ayrı fonksiyon çağrısı vardır, ama bunlar aynı anda sonlanmaz; her seferinde biri önce biter. Bu yüzden backward substitution karışık hâle gelir. Ağaç kurmak burada kolaylık sağlar. n için $n-1$ ve $n-2$'ye çağrı yapılır ve her çağrıda sabit bir işlem (toplama) vardır. $n-1$ düğümü $n-2$ ve $n-3$'e, $n-2$ düğümü $n-3$ ve $n-4$'e çağrı yapar. Örneğin $n = 5$ iken bir dal daha aşağı inerken diğer dal daha önce biter. Ağaç tam dolu değildir.

Ağaç tam dolu olsaydı, Hanoi'deki gibi bir görüntü olurdu; düğüm sayısı gerçekten işlem sayısına karşılık gelir ve üst sınır olarak 2^n verirdi. Gerçekten de karmaşıklık 2^n'den aşağıdadır, çünkü daha önce sonlanan çağrılar vardır. Hoca ardışık Fibonacci sayılarının oranının altın oranı verdiğini hatırlattı ve 2 yerine altın oran konduğunda karmaşıklığın $\Theta(1.618^n)$'e karşılık geldiğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Altın oran $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$'dir ve $x^2 = x + 1$ denkleminin pozitif köküdür. Fibonacci recurrence'ının çözümü bu denklemden gelir; bu yüzden $F(n)$ ve özyinelemeli çağrı sayısı φ^n mertebesinde büyür.

12. Alıştırmalar

Dersin son kısmında hoca konuyu pekiştirmek için alıştırmalar çözdü. Bu alıştırmaların soru metinleri ve çözümleri aşağıdaki "Örnek Sorular" bölümünde ayrıntılı verilmiştir; burada yalnız hocanın vurguladığı noktalar toplanmıştır.

- Katsayıların önemsiz sayılmasının nedeni: $100n^2$ ile $0.01n^3$'te n'in küçük değerlerinde fark görülmez, ama büyük n'de biri kübik, diğeri karesel davranır. Bu yüzden sınırların n_0'ın hangi değerinden itibaren geçerli olduğu bilgisi önemlidir.
- Sabit katsayılı iyileştirmeler verimlilik sınıfını değiştirmez. Secret algoritmasında `if` yerine `else if` kullanmak bir tık zaman kazandırır, ama $\Theta(n)$ değişmez.
- Simetrik matris kontrolünde transpozu alıp karşılaştırmak daha iyi değildir: transpoz için zaten n^2 gerekir, karşılaştırma eklenince sınıf değişmez ama iş iki katına çıkar.
- Gerçek iyileştirme sınıfı değiştirir: Mystery'de döngü yerine kapalı formül kullanmak $\Theta(n)$'den sabit zamana iner.

Örnek Sorular

Soru 1: Girdi iki katına çıkınca süre

(ekrandan, 00:15:20) $C(n) = \frac{1}{2}n(n-1)$ olduğu varsayılırsa, girdi boyutu iki katına çıkarıldığında algoritma ne kadar uzun çalışır?

Çözüm (derste verildi): $C(n) \approx \frac{1}{2}n^2$. $\frac{T(2n)}{T(n)} \approx \frac{\frac{1}{2}(2n)^2}{\frac{1}{2}n^2} = 4$. Yaklaşık 4 kat.

Soru 2: Big Omega aynı mertebeyi dışlar mı?

(ekrandan, 01:11:40) Big Omega notasyonu, $n!$ ile 2^n'in aynı büyüme mertebesinde olma ihtimalini dışlayabilir mi?

Çözüm (derste verildi): Hayır. Ω'daki "büyük eşit" eşitlik durumunu da kapsar. Limit yöntemi ise dışlar: $\lim n!/2^n = \infty$ çıktığı için aynı mertebede olamazlar (aynı mertebede olsalardı limit sabit olurdu).

Soru 3: Büyüme hızı karşılaştırmaları

(derste sözlü, 01:44:43) Aşağıdaki fonksiyon çiftlerinin büyüme hızlarını karşılaştırın.

- (a) $n(n+1)$ ve $2000n^2$
- (b) $100n^2$ ve $0.01n^3$
- (c) $\log_2 n$ ve $\ln n$
- (d) $(n-1)!$ ve $n!$

Çözüm (derste verildi):

- (a) $n(n+1) = n^2 + n$; baskın terim ikisinde de n^2'dir, aynı büyüme mertebesi.
- (b) $0.01n^3$ kübik, $100n^2$ karesel; küçük n'de fark görünmese de $0.01n^3$ daha hızlı büyür.
- (c) Taban değiştirilerek $\log_2 n = \frac{1}{\ln 2}\ln n$; aradaki fark sabit katsayıdır, aynı mertebe.
- (d) $n! = n \cdot (n-1)!$; oran n ile sonsuza gider, $n!$ daha hızlı büyür.

Soru 4: Toplamın büyüme mertebesi

(ekrandan, 01:47:55) Aşağıdaki toplamın büyüme mertebesini, mümkün olan en basit $g(n)$ ile $\Theta(g(n))$ olarak bulun:

$$\sum_{i=0}^{n-1} (i^2+1)^2$$

Çözüm (derste verildi): Kullanılan formül $\sum_{i=1}^{n} i^k \approx \frac{1}{k+1}n^{k+1}$.

$$\sum_{i=0}^{n-1}(i^2+1)^2 = \sum_{i=0}^{n-1}(i^4+2i^2+1) \approx \sum_{i=0}^{n-1} i^4$$

Formül $i = 1$'den n'e kadar olduğu için $i = 0$ terimi eklenir, $i = n$ terimi çıkarılır:

$$\sum_{i=0}^{n-1} i^4 = \sum_{i=1}^{n} i^4 + 0^4 - n^4 = \sum_{i=1}^{n} i^4 - n^4 \approx \frac{1}{4+1}n^{4+1} - n^4 = \frac{1}{5}n^5 - n^4 \approx \frac{1}{5}n^5 \in \Theta(n^5)$$

Hoca bunu bir polinom gibi düşünmeyi önerdi: en yüksek dereceli terim Θ karşılığını verir. (Python ile $n = 1000$ için toplam$/n^5 \approx 0.1995$, yani $\frac{1}{5}$'e yaklaşıyor.)

Soru 5: Mystery algoritması

(ekrandan, 01:49:00)

```
ALGORITHM Mystery(n)
//Input: A nonnegative integer n
S <- 0
for i <- 1 to n do
    S <- S + i * i
return S
```

Bu algoritma ne hesaplar? Temel işlemi nedir? Temel işlem kaç kez çalışır? Verimlilik sınıfı nedir? Olası bir iyileştirme var mı?

Çözüm (derste verildi):

- Ne hesaplar: 1'den n'e kadar sayıların karelerinin toplamını, $S = \sum_{i=1}^{n} i^2$.
- Temel işlem: çarpma (toplama da aynı sayıda çalışır).
- Kaç kez: n kez.
- Verimlilik sınıfı: $\Theta(n)$.
- İyileştirme: Döngüye gerek yoktur; kareler toplamının kapalı formülü kullanılarak sabit zamanda, $O(1)$'de çözülür.

  Ders sonrası not: Hoca formülü "n çarpı $n+1$ çarpı $2n+1$" diye söyledi; doğrusu $\frac{n(n+1)(2n+1)}{6}$'dır. Örneğin $n = 10$ için döngü ve formül ikisi de 385 verir (python ile kontrol edildi).

Soru 6: Secret algoritması

(ekrandan, 01:50:00)

```
ALGORITHM Secret(A[0..n-1])
//Input: An array A[0..n-1] of n real numbers
minval <- A[0]; maxval <- A[0]
for i <- 1 to n-1 do
    if A[i] < minval
        minval <- A[i]
    if A[i] > maxval
        maxval <- A[i]
return maxval - minval
```

Bu algoritma ne hesaplar? Temel işlemi nedir? Temel işlem kaç kez çalışır? Verimlilik sınıfı nedir? Olası bir iyileştirme var mı?

Çözüm (derste verildi):

- Ne hesaplar: Dizideki en büyük ve en küçük elemanın farkını.

- Temel işlem: karşılaştırma. Atamalar her seferinde yapılmaz, karşılaştırmalar her seferinde yapılır.
- Kaç kez: Döngü başına iki karşılaştırma, toplam $2(n-1)$.
- Verimlilik sınıfı: Sabit katsayı ihmal edildiği için $\Theta(n)$.
- İyileştirme: İkinci `if` yerine `else if` yazılabilir; böylece eleman minimumdan küçükse maksimum kontrolü yapılmaz. Bu biraz zaman kazandırır ama verimlilik sınıfını değiştiren bir iyileştirme değildir.

  Ek bilgi (derste anlatılmadı): `else if` ile en kötü durumda (örneğin artan sıralı dizide) karşılaştırma sayısı yine $2(n-1)$'dir. Elemanları ikişer ikişer alıp önce kendi aralarında karşılaştıran bilinen yöntem karşılaştırma sayısını yaklaşık $\frac{3n}{2}$'ye indirir; bu da yalnız sabiti değiştirir, sınıf $\Theta(n)$ kalır.

Soru 7: Simetrik matris kontrolü

(derste sözlü, 01:51:49) Bir kare matrisin simetrik olup olmadığını, karşılıklı elemanları ($A[1][2]$ ile $A[2][1]$, $A[1][3]$ ile $A[3][1]$ gibi) karşılaştırarak kontrol eden algoritmanın temel işlemi, çalışma sayısı ve verimlilik sınıfı nedir? İyileştirilebilir mi?

Çözüm (derste verildi): Temel işlem karşılaştırmadır. İç döngü $j = i+1..n-1$, dış döngü $i = 0..n-2$:

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \frac{(n-1)n}{2} \in \Theta(n^2)$$

Hoca transpozu alıp üst üste karşılaştırma fikrini tartıştı: transpoz için zaten n^2'lik bir karmaşıklık gerekir, karşılaştırmalar da eklenince verimlilik sınıfı değişmez ama yapılan iş yaklaşık iki katına çıkar. Yani bu daha iyi bir yol değildir.

Soru 8: Izgarada büyüyen şekil

(ekrandan, 01:56:40) Bir ızgaranın ortasında $n = 0$'da tek bir kare vardır. Her adımda boyalı karelerin dört komşusu boyanır ($n = 1$'de artı biçimli 5 kare, $n = 2$'de 13 kare). n. adımdaki kare sayısı $C(n)$ ve karmaşıklığı nedir?

Çözüm (derste verildi): Slayttaki değerler:

- $C(0) = 1$
- $C(1) = 5 = 1 + 4 = 1 + 4 \cdot 1 = C(0) + 4 \cdot 1$
- $C(2) = 13 = 5 + 8 = 5 + 4 \cdot 2 = C(1) + 4 \cdot 2$
- $C(3) = 25 = 13 + 12 = 13 + 4 \cdot 3 = C(2) + 4 \cdot 3$

Recurrence: $C(n) = C(n-1) + 4n$, $C(0) = 1$. Backward substitution: $C(n-1) = C(n-2) + 4(n-1)$, $C(n-2) = C(n-3) + 4(n-2)$, ... yerine konarak

$$C(n) = C(n-i) + 4(n-i+1) + \cdots + 4(n-1) + 4n$$

$C(0)$'a ulaşmak için $i = n$:

$$C(n) = C(0)+4\cdot 1+4\cdot 2+\cdots+4n = 1+4(1+2+\cdots+n) = 1+4\cdot\frac{n(n+1)}{2} = 1+2n(n+1) = 2n^2+2n+1$$

Sonuç $\Theta(n^2)$'dir. ($n = 0..4$ için recurrence ve kapalı form python ile karşılaştırıldı: 1, 5, 13, 25, 41.)

> **Ders sonrası not:** Slayt recurrence için "for all $n \geq 0$" yazıyor; $C(0) = 1$ başlangıç değeri olduğu için recurrence büyük olasılıkla $n \geq 1$ için kastedilmiştir.

Dersin Toparlaması

Bu derste algoritma analizinin temel araçları kuruldu. Çalışma zamanı $T(n) \approx c_{op}C(n)$ ile modellendi ve sisteme bağlı c_{op}'nin büyüme mertebesi açısından önemsiz olduğu, asıl belirleyicinin $n \to \infty$'daki davranış olduğu gösterildi. Büyüme mertebeleri tablosu, en kötü, en iyi ve ortalama durum analizi (sıralı arama ve p olasılığı), Big O, Big Omega ve Big Theta'nın tanımları ve sabit bulma örnekleri, ardışık parçalar teoremi ve limitle karşılaştırma (Stirling formülü dahil) işlendi.

İkinci yarıda özyinelemesiz algoritmalar toplam sembolleriyle (MaxElement, UniqueElements, matris çarpımı, ikilik basamak sayısı), özyinelemeli algoritmalar ise recurrence relation ve backward substitution ile (faktöriyel, Hanoi kuleleri, özyinelemeli ikilik basamak sayısı) analiz edildi. Çağrı ağacı yöntemi gösterildi ve Fibonacci örneğinde backward substitution'ın zorlandığı, karmaşıklığın altın oran üzeri n mertebesinde olduğu anlatıldı. Hoca, her çağrıda birden fazla işlem yapılan recurrence'ların ağaçla çözümüne ilerleyen haftalarda döneceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.